1. Author

- Name: Shreya Dnyandeep Pazare
- Roll Number: 24f1000077
- Email: 24f1000077@ds.study.iitm.ac.in
- About Me: I am a student in the IIT Madras BS Degree program with a deep interest in developing performant web applications and scalable systems. I enjoy building scalable platforms like the Central University Portal and working on technical challenges that bridge theoretical concepts with real-world utility.

2. Project Details

Problem Statement:
Institutes need an easy way to manage campus placements. The system connects students and hiring companies, replacing manual spreadsheets and emails. It handles company sign-ups, job postings, student applications, and interview scheduling.

Approach:
I built a web application with three user roles: Admin, Company, and Student. Admin approves companies and job postings. Companies post jobs, review applications, and schedule interviews. Students view approved placement drives and apply to them. The backend handles data storage and processing, while the frontend provides the user interface. A memory cache (Redis) speeds up searches, and a task queue (Celery) sends reminder emails and exports application history in the background.

3. AI/LLM Declaration

I used AI assistants to help with documentation, suggest ways to organize the database, and plan backend connection points. I wrote the final code, designed the database tables, and tested the integration manually. Total AI assistant usage on this project was approximately 20%.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Python / Flask	Backend server and request handling
Vue	Interactive frontend pages
Bootstrap	Responsive layout for mobile and desktop
SQLite	Database storage
SQLAlchemy	Database interaction from backend code
Redis	Caching search results and background task queue
Celery	Background tasks — reminder emails and history exports
Axios	Frontend-to-backend HTTP communication
Flask-JWT-Extended	JWT-based authentication and role-based route protection
Flask-Mail	Sends the reminder and monthly-report emails
Flask-Caching	Redis-backed response caching with per-key expiry
Flask-Cors	Allows the Vue frontend to call the Flask API cross-origin

5. Database Schema / ER Diagram

Tables:

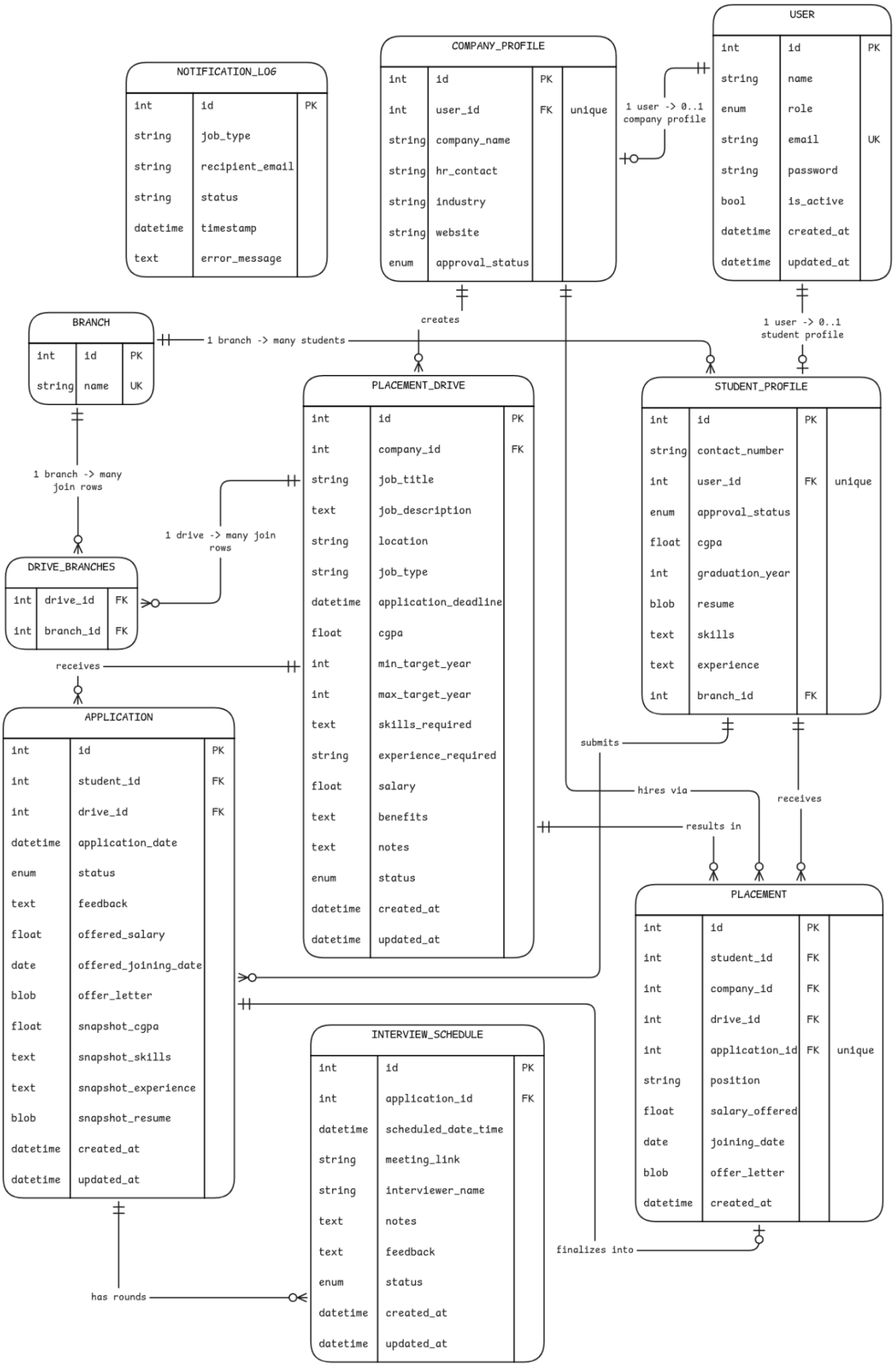
- **User** - stores user credentials and role information (id, name, email, password, role, is_active, created_at, updated_at)
- **Branch** - stores academic department names (id, name); 8 seeded by default (CS, IT, ECE, EE, ME, Civil, Chemical, Biotech)
- **CompanyProfile** - stores company registration and approval details (id, user_id, company_name, hr_contact, industry, website, approval_status)
- **StudentProfile** - stores student academic and application details (id, user_id, contact_number, approval_status, cgpa, graduation_year, resume, skills, experience, branch_id)
- **PlacementDrive** - stores job postings created by companies (id, company_id, job_title, job_description, location, job_type, application_deadline, cgpa, min_target_year, max_target_year, skills_required, experience_required, salary, benefits, notes, status, created_at, updated_at)
- **drive_branches** - join table linking a PlacementDrive to every Branch it is open to (drive_id, branch_id); a drive with no rows here is open to all branches
- **Application** - stores student applications to placement drives, including a point-in-time snapshot of the applicant's profile taken at the moment of applying (id, student_id, drive_id, application_date, status, feedback, offered_salary, offered_joining_date, offer_letter, snapshot_cgpa, snapshot_skills, snapshot_experience, snapshot_resume, created_at, updated_at); unique constraint on (student_id, drive_id) enforces one application per student per drive
- **InterviewSchedule** - stores interview rounds scheduled for an application (id, application_id, scheduled_date_time, meeting_link, interviewer_name, notes, feedback, status, created_at, updated_at)
- **Placement** - stores the final placement record created once a student accepts an offer (id, student_id, company_id, drive_id, application_id, position, salary_offered, joining_date, offer_letter, created_at)
- **NotificationLog** - audit trail of every background email sent by Celery (id, job_type, recipient_email, status, timestamp, error_message)

Relationships:

- One-to-One → User → CompanyProfile (each company has one user account)
- One-to-One → User → StudentProfile (each student has one user account; the Admin account has neither)
- One-to-Many → Branch → StudentProfile (one branch has many students)
- Many-to-Many → Branch → PlacementDrive (through drive_branches; a drive can be restricted to several branches, and a branch can be eligible for several drives)
- One-to-Many → CompanyProfile → PlacementDrive (one company creates many placement drives)
- One-to-Many → CompanyProfile → Placement (one company accumulates many placement records)
- One-to-Many → StudentProfile → Application (one student submits many applications)
- One-to-Many → StudentProfile → Placement (one student can accumulate multiple placement records, one per accepted offer, across different drives)
- One-to-Many → PlacementDrive → Application (one drive receives many applications)
- One-to-Many → PlacementDrive → Placement (one drive can result in many placement records)
- One-to-Many → Application → InterviewSchedule (one application can have multiple interview rounds)

- One-to-One → Application → Placement (one application finalizes into exactly one placement once its offer is accepted)
- Standalone → NotificationLog (records what was sent and to whom but holds no foreign keys to other tables)

Entity-Relationship Diagram:



6. API Resource Endpoints

User Login and Registration:

- GET /api/auth/branches - List academic branches
- POST /api/auth/register/student - Create a student account
- POST /api/auth/register/company - Create a company account
- POST /api/auth/login - Authenticate and return an access token
- GET /api/auth/me - Get the logged-in user's details

Admin:

- GET /api/admin/overview - Student, approved-company, and approved-drive counts
- GET /api/admin/companies - List companies, with search and filters
- POST /api/admin/companies - Create a company directly (pre-approved)
- PATCH /api/admin/companies/<id> - Edit company profile
- POST /api/admin/companies/<id>/approve - Approve a company registration
- POST /api/admin/companies/<id>/reject - Reject a company registration
- POST /api/admin/companies/<id>/blacklist - Deactivate a company and reject its active drives
- POST /api/admin/companies/<id>/whitelist - Reactivate a blacklisted company
- GET /api/admin/drives - List placement drives, with search and filters (company, status, skills, CGPA, branch, deadline)
- POST /api/admin/drives - Create a placement drive directly (pre-approved)
- PATCH /api/admin/drives/<id> - Edit a placement drive
- POST /api/admin/drives/<id>/approve - Approve a placement drive
- POST /api/admin/drives/<id>/reject - Reject a placement drive
- POST /api/admin/drives/<id>/close - Stop accepting new applications
- POST /api/admin/drives/<id>/cancel - Cancel a drive and reject active applications
- POST /api/admin/drives/<id>/extend-deadline - Change the application deadline
- GET /api/admin/applications - List applications, with filters
- GET /api/admin/applications/<id> - Get application details
- PATCH /api/admin/applications/<id> - Update application status/notes
- GET /api/admin/students - List students, with search and filters
- PATCH /api/admin/students/<id> - Edit student profile
- POST /api/admin/students/<id>/blacklist - Deactivate a student and withdraw active applications
- POST /api/admin/students/<id>/whitelist - Reactivate a blacklisted student
- GET /api/admin/students/<id>/resume - Download a student's resume
- GET /api/admin/applications/<id>/resume - Download the resume submitted for an application
- GET /api/admin/applications/<id>/offer-letter - Download an application's offer letter
- GET /api/admin/interviews - List all scheduled interviews
- PATCH /api/admin/interviews/<id> - Edit interview date, status, or notes

Company:

- GET /api/company/overview - Company details and its placement drives
- GET /api/company/drives - List drives created by the company
- POST /api/company/drives - Create a new placement drive
- PATCH /api/company/drives/<id> - Edit a pending or rejected drive
- POST /api/company/drives/<id>/extend-deadline - Change the application deadline
- POST /api/company/drives/<id>/close - Stop accepting new applications
- POST /api/company/drives/<id>/cancel - Cancel a placement drive
- GET /api/company/applications - List applicants to the company's drives
- GET /api/company/applications/<id> - Get applicant details

- PATCH /api/company/applications/<id>/status - Advance or reject an applicant
- POST /api/company/applications/<id>/schedule-interview - Schedule an interview
- POST /api/company/applications/<id>/offer - Upload an offer letter and salary details
- GET /api/company/applications/<id>/offer-letter - Download the uploaded offer letter
- GET /api/company/applications/<id>/resume - View a candidate's resume
- GET /api/company/interviews - List interviews scheduled by the company
- GET /api/company/profile - Get company profile
- PATCH /api/company/profile - Edit profile or change password

Student:

- GET /api/student/drives - List approved placement drives
- GET /api/student/companies - List companies with approved drives
- POST /api/student/drives/<id>/apply - Apply to a placement drive
- GET /api/student/applications - List the student's own applications
- GET /api/student/applications/<id> - Get application status and details
- POST /api/student/applications/<id>/withdraw - Withdraw an application
- POST /api/student/applications/<id>/accept-offer - Accept an offer and finalize placement
- GET /api/student/applications/<id>/offer-letter - View the received offer letter
- GET /api/student/applications/<id>/resume - View the resume snapshot submitted with an application
- GET /api/student/profile - Get profile details
- PATCH /api/student/profile - Edit profile, upload resume, or change password
- GET /api/student/profile/resume - Download the current resume
- POST /api/student/applications/export - Start a background export of application history
- GET /api/student/applications/export/<task_id> - Check export status
- GET /api/student/applications/export/<task_id>/download - Download the exported history file
- GET /api/student/events - Server-Sent Events stream of live application/interview status updates

7.Architecture and Features:

Project Structure:

```
24f1000077/
├── backend/                # Flask backend application
│   ├── app.py              # Flask application entry point and configuration setup
│   ├── config.py           # App configuration settings (Database, Redis, JWT, etc.)
│   ├── models.py           # SQLAlchemy database models and relationships
│   ├── services/           # Flask Blueprints for auth, admin, company, and student routes
│   ├── serializers.py      # Object serialization logic for JSON API responses
│   ├── tasks.py            # Celery background tasks
│   ├── celery_workers.py   # Celery worker configuration and entry point
│   ├── utils.py            # Helper utilities and shared backend logic
│   ├── realtime.py         # Publishes application-status change events to Redis pub/sub for live updates
│   ├── seed.py             # Database initialization and dummy data generation
│   ├── requirements.txt    # Python dependencies for the backend
│   └── instance/           # SQLite database storage
├── frontend/               # Vue 3 + Vite Single Page Application (UI)
│   ├── src/                # Main Vue source code
│   │   ├── components/     # Reusable UI components (modals, layout, badges)
│   │   ├── views/          # Dashboard page views for Admin, Company, and Student
```

```
│   │   ├── router/          # Vue Router configuration for client-side navigation
│   │   ├── services/        # API client setup and Axios interceptors
│   │   ├── utils/           # Frontend helper utilities
│   │   ├── assets/          # Global CSS stylesheets and static assets
│   │   ├── App.vue          # Root Vue component
│   │   └── main.js          # Vue application entry point
│   ├── package.json         # Node dependencies and npm scripts
│   └── vite.config.js       # Vite bundler configuration
└── README.md                # Setup instructions, architecture overview, and troubleshooting
```

Implemented Features:

User registration and login (JWT-based authentication)
Role-based access control (Admin, Company, Student dashboards)
Company approvals and placement drive management
Student application tracking and hiring management
Search, filtering (by Branch, CGPA, Graduation Year), and sorting views
Secure password hashing and live active-session validation
Asynchronous CSV export of student applications
Background Celery jobs for daily deadline reminders and monthly reports
Redis-backed API caching for performance optimization

Additional Features:

Admin functionality to create company and placement drives.
Application point-in-time snapshotting (CGPA, Resume, Skills)

8. Video Presentation

The presentation and demo video can be accessed using the following link:

PROJECT_DEMO_VIDEO.mp4